

Pallimasin

Madina on IOI osalejate lõbustamiseks leiutanud pallimasina. Masina sisemine struktuur on järgmine:

- Masin koosneb N **tipust**, millel on indeksid 0 kuni $N - 1$. Tippu $N - 1$ nimetatakse **juureks**.
- Iga tipu u puhul, kus $0 \leq u < N - 1$, on tipu u **ülem** tipp $P[u]$, kus $P[u] > u$, ja tipp u on $P[u]$ **alam**. Juurel puudub ülem. Tippu, millel puuduvad alamad, nimetatakse **leheks**.

Sa **ei tea** arvu N ega ühegi tipu u ülemat $P[u]$. Selle asemel ütleb Madina sulle lehtede arvu M , ja et lehtede indeksid on $0, 1, \dots, M - 1$. Sinu ülesanne on masinaga töötades leida selle sisemine struktuur.

Masina igas tipus saab olla ülimalt üks **pall**. Igal pallil on **väärtus**, mis on mittenegatiivne täisarv. Me ütleme, et tipu väärtus on x , kui selles on pall väärtusega x . Tipp on **hõivatud**, kui selles on pall, vastasel juhul on see **vaba**. Algselt on iga tipp vaba.

Saad välja kutsuda kaht funktsiooni:

- **insert**(U, X): Proovi sisestada lehte U pall väärtusega X .
 - Kui leht U on hõivatud, tagastab funktsioon väärtuse `false` ilma palli sisestamata.
 - Kui leht U on vaba, sisestatakse pall lehte U . Seejärel liigub pall korduvalt oma praeguse tipu ülemtippu, kui see ülem on vaba. Liikumine peatub, kui pall jõuab juureni või tippu, mille ülem on hõivatud. Funktsioon tagastab `true`.
- **collect**(): Kui juur on vaba (mis tähendab, et masin on tühi), tagastab `collect` tühja massiivi. Vastasel juhul kogub allpool kirjeldatud rekursiivne funktsioon `traverse` kõigi masinas olevate pallide väärtused massiivi S järgmiselt:

Algselt on massiiv S tühi ja kutsutakse välja `traverse(N - 1)`.

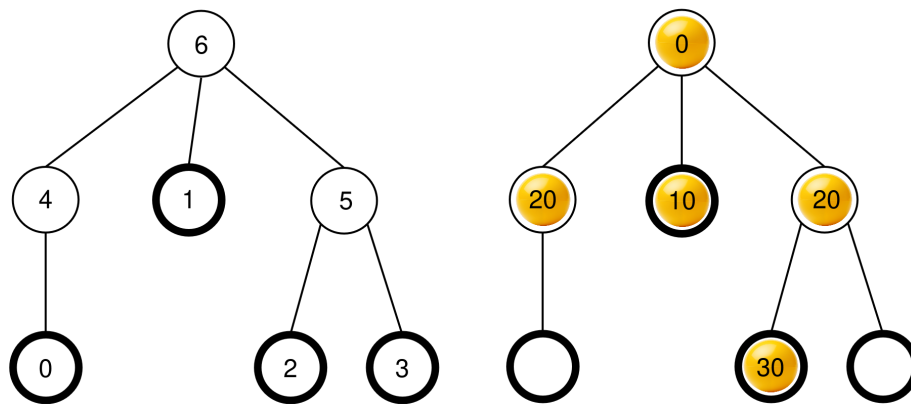
```

traverse(u):
    lisa tipus u oleva palli väärtus massiivi S lõppu
    olgu c[u] tipu u hõivatud alamate loend
    sorteer c[u] selle tippudes olevate pallide väärtuste järgi
        mittekahanevasse järjekorda (kui mitmes tipus on sama väärtus,
        võivad nad olla mistahes järjekorras)
    for each v in c[u]:
        traverse(v)

```

Pärast selle funktsiooni töö lõppu **eemaldatakse** masinast **kõik pallid** ja `collect` tagastab massiivi *S*.

Vaatame näitena masinat, millel on $N = 7$ tippu ja millel on ülemate massiiv $P = [4, 6, 5, 5, 6, 6]$. Vasakpoolne pilt näitab tühja masinat koos tippude indeksitega, parempoolne pilt aga pallide võimalikku konfiguratsiooni pärast üht `insert`-operatsioonide jada (toodud näidete osas).



Kui kutsutakse välja `collect()`, on juur (tipp 6) hõivatud, seega *S* initsialiseeritakse kujul $S = []$ ja kutsutakse välja `traverse(6)`.

traverse(6): Tipu 6 väärtus on 0, selle lisamine massiivile *S* annab tulemuseks $S = [0]$. Tipu 6 alamad on 4, 1 ja 5, mis kõik on hõivatud. Nende väärtused on vastavalt 20, 10 ja 20. Mittekahanevas järjekorras sortimine annab tulemuseks $c[6] = [1, 5, 4]$ (pane tähele, et $c[6] = [1, 4, 5]$ oleks samuti lubatud, kuna tippudel 4 ja 5 on sama väärtus). Seejärel kutsutakse välja funktsiooni `traverse` iga $c[6]$ tipu jaoks selles järjekorras.

- **traverse(1):** Tipu 1 väärtus on 10; selle lisamine massiivi *S* annab tulemuseks $S = [0, 10]$. Tipul 1 pole hõivatud alamaid, seega funktsioon lõpetab töö.
- **traverse(5):** Tipu 5 väärtus on 20; selle lisamine massiivi *S* annab tulemuseks $S = [0, 10, 20]$. Tipul 5 on üks hõivatud alamtipp, tipp 2, seega $c[5] = [2]$ ja kutsutakse välja `traverse(2)`.
 - **traverse(2):** Tipu 2 väärtus on 30; selle lisamine massiivi *S* annab tulemuseks $S = [0, 10, 20, 30]$. Tipul 2 pole hõivatud alamtippe, seega funktsioon lõpetab töö.

- Täitmine naaseb `traverse(5)` juurde, mis on töödelnud kõik $c[5]$ elemendid, seega funktsioon lõpetab töö.
- **`traverse(4)`**: Tipu 4 väärtus on 20; selle lisamine massiivi S annab tulemuseks $S = [0, 10, 20, 30, 20]$. Tipul 4 pole hõivatud alamtippe, seega funktsioon lõpetab töö.

Kõik väljakutsed on lõpule viidud ja rohkem tippe pole vaja töödelda. Lõpuks võetakse kõik pallid masinast välja ja `collect` tagastab massiivi $S = [0, 10, 20, 30, 20]$.

Sinu ülesanne on leida tippude arv N ja esitada ülemate massiiv $R = [R[0], R[1], \dots, R[N-2]]$, mis kirjeldab masina struktuuri, kuid tippudel, mis ei ole lehed ega juur, võib olla erinev tähistus. Formaalselt loetakse esitatud massiiv R korrektseks, kui igale masina tipule u on võimalik määrata erinev märgend $L[u]$ väärtusega $0 \leq L[u] < N$ nii, et:

- $L[u] = u$ kõigi $0 \leq u < M$ ja $u = N - 1$ korral ning
- $L[P[u]] = R[L[u]]$ kõigi $0 \leq u < N - 1$ korral.

Pole nõutav, et $R[u] > u$. Masin peab lahenduse esitamise ajal **olema tühi**.

Olgu K sooritatud `collect` operatsioonide arv ja B suurim väärtus, mida on kasutatud mistahes pallil kõigi `insert` operatsioonide käigus. Olgu $C = K + B$.

C ei tohi ületada 1000. Mõnes alamülesandes sõltub punktisumma C väärtusest.

Realiseerimine

Sul tuleb kirjutada järgmine funktsioon:

```
std::vector<int> find_structure(int M)
```

- M : masina lehtede arv.
- Seda funktsiooni kutsutakse igas testis välja täpselt üks kord.
- Funktsioon peab tagastama massiivi $R = [R[0], R[1], \dots, R[N-2]]$ pikkusega $N - 1$, mis on korrektne ülemate massiiv.

Masinaga suhtlemiseks võib sinu funktsioon kutsuda välja kahte funktsiooni.

Esimene funktsioon on:

```
bool insert(int U, int X)
```

- U : Lehe indeks, kuhu pall tuleks sisestada. Peab olema täidetud tingimus $0 \leq U < M$.
- X : Palli täisarvuline väärtus. Peab olema täidetud tingimus $0 \leq X \leq 1000$.
- Funktsioon tagastab väärtuse `true`, kui pall sisestati edukalt, või `false` kui leht U oli juba hõivatud.

- Funktsiooni võib välja kutsuda maksimaalselt 500 000 korda.

Teine funktsioon on:

```
std::vector<int> collect()
```

- Kogub kõigi sisestatud pallide väärtused, tühjendab masina ja tagastab kogutud massiivi S .

Kui programmi täitmise ajal ületab C väärtus mingil hetkel 1000, saab lahendus hinnanguks
Output isn't correct: Too many resources used.

Masina struktuur fikseeritakse enne `find_structure` kutsumist. Hindaja on **deterministlik** selles mõttes, et kui mitu korda sooritada sama operatsioonide jada, tagastab `collect` alati samasugused massiivid.

Piirangud

- $2 \leq N \leq 1000$.
- $1 \leq M \leq 200$.
- $M < N$.

Alamülesanded

Alamülesanne	Skoor	Lisapiirangud
1	5	Juurel on täpselt M alamat.
2	10	$M \leq 3$.
3	25	$N \leq 200, M \leq 45$.
4	60	Lisapiiranguid pole.

Alamülesannetes 3 ja 4 sõltub teie tulemus C väärtusest järgmiselt.

Alamülesanne 3 (25 punkti)

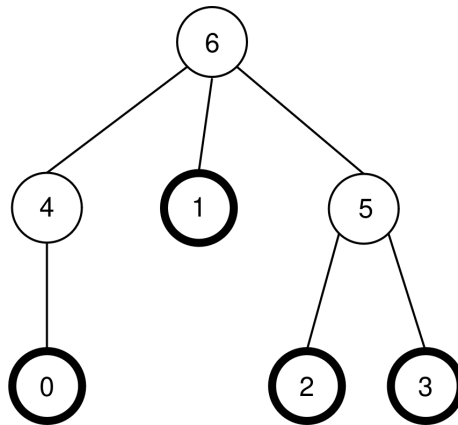
Piirangud	Skoor
$1000 < C$	0
$45 < C \leq 1000$	13
$C \leq 45$	25

Alamülesanne 4 (60 punkti)

Piirangud	Skoor
$1000 < C$	0
$200 < C \leq 1000$	7
$71 < C \leq 200$	$47 - \frac{C}{5}$
$44 < C \leq 71$	$104 - C$
$C \leq 44$	60

Näide

Vaatleme sama masinat nagu ülesande kirjelduses, seega $N = 7$, $M = 4$ ja $P = [4, 6, 5, 5, 6, 6]$. Masinat on taas kujutatud järgmisel joonisel:



Hindaja teeb järgmise väljakutse:

```
find_structure(4)
```

Funktsioon võib teha näiteks järgmised väljakutsed:

1. **insert(0, 0):** Leht 0 on vaba, seega sisestatakse lehte 0 pall väärtusega 0. Tipp $P[0] = 4$ on vaba, seega liigub pall tippu 4. Tipp $P[4] = 6$ on vaba, seega liigub pall tippu 6. Kuna tipp 6 on juur, siis liikumine peatub. Väljakutse tagastab `true`.
2. **insert(3, 20):** Leht 3 on vaba, seega sisestatakse lehte 3 pall väärtusega 20. Tipp $P[3] = 5$ on vaba, seega liigub pall tippu 5. Kuna $P[5] = 6$ on hõivatud, siis liikumine peatub. Väljakutse tagastab `true`.
3. **insert(1, 10):** Leht 1 on vaba, seega sisestatakse lehte 1 pall väärtusega 10. Kuna $P[1] = 6$ on hõivatud, jääb pall tippu 1. Väljakutse tagastab `true`.
4. **insert(2, 30):** Leht 2 on vaba, seega sisestatakse lehte 2 pall väärtusega 30. Kuna $P[2] = 5$ on hõivatud, jääb pall tippu 2. Väljakutse tagastab `true`.

5. `insert(1, 25)`: Leht 1 on hõivatud, seega palli lehte 1 ei sisestata. Väljakutse tagastab `false`.
6. `insert(0, 20)`: Leht 0 on vaba, seega sisestatakse lehte 0 pall väärtusega 20. Tipp $P[0] = 4$ on vaba, seega liigub pall tippu 4. Kuna $P[4] = 6$ on hõivatud, siis liikumine peatub. Väljakutse tagastab `true`.

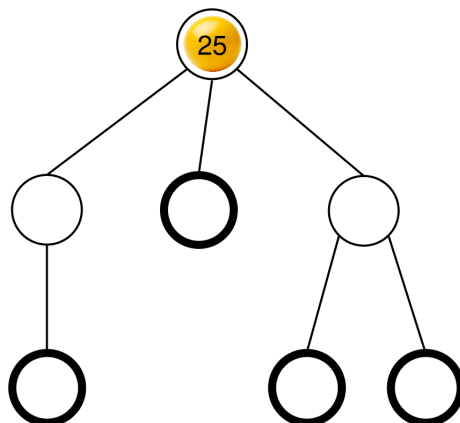
Tulemusena saadud pallide konfiguratsioon oli juba ülesande kirjelduses kuvatud.

Järgmisena võib kutsuda välja funktsiooni

```
collect()
```

Selle väljakutse täitmist selgitati juba ülesande kirjelduses, seega tagastab funktsioon massiivi $S = [0, 10, 20, 30, 20]$. Pärast seda on masin jälle tühi.

Järgmiseks võib välja kutsuda näiteks funktsiooni `insert(2,25)`. Leht 2 on vaba, seega sisestatakse lehte 2 pall väärtusega 25. Pall liigub seejärel tippu 5 ja siis tippu 6, kus see peatub. Väljakutse tagastab `true`. Saadud pallide konfiguratsioon on näidatud järgmisel joonisel.



Siis võib välja kutsuda funktsiooni `collect()`, mis tagastab $S = [25]$ ja tühjendab masina.

Funktsioon `find_structure` võib tagastada kaks korrektset ülemate massiivi: $R = P = [4, 6, 5, 5, 6, 6]$ (mis vastab märgendusele $L = [0, 1, 2, 3, 4, 5, 6]$) ja $R = [5, 6, 4, 4, 6, 6]$ (mis vastab märgendusele $L = [0, 1, 2, 3, 5, 4, 6]$). Selles näites $K = 2$ ja $B = 30$, seega $C = 32$.

Näidishindaja

Sisendi vorming:

```
N M
P[0] P[1] P[2] ... P[N-2]
```

Väljundi vorming:

```
K B C
```

```
Q
```

```
R[0] R[1] R[2] ... R[Q-1]
```

Siin on Q funktsiooni `find_structure` tagastatud massiivi R pikkus.